

Shell Script – Chuleta Completa

¿Qué es un Script?

- Archivo con comandos que el sistema ejecuta secuencialmente.
- Automatiza tareas: copias, configuraciones, instalaciones, etc.

Shebang (#!/bin/bash)

- Indica qué intérprete debe usarse.
Ejemplos:
- #!/bin/sh,
- #!/usr/bin/python3,
- #!/usr/bin/env node

Comentarios

- Se escriben con # y no se ejecutan.
- Pueden ir en su propia línea o al final de un comando.

VARIABLES

Asignación:

```
nombre="Maria"
(Sin espacios alrededor de =)
```

Uso:

```
echo "Hola $nombre"
echo "Hola ${nombre}"
```

Características:

- Sin tipo (todo es string)
- Case sensitive
- Pueden empezar con letra o guion bajo (_)

Escape de caracteres:

```
echo hola \> a # muestra
literalmente "hola > a"
```

Comillas:

- "dobles": expansión de variables
- 'simples': valor literal

Leer del usuario:

```
read -p "Tu nombre: " nombre
```

Guardar salida de comando:

```
listado=$(ls -la ~)
# o también
listado=`ls -la ~`
```

VARIABLES DE ENTORNO

Variable	Significado
\$HOME	Carpeta del usuario

\$PATH	Rutas de búsqueda de comandos
\$UID	ID del usuario actual
\$HOSTNAME	Nombre del equipo

🔗 Ver todas:
env

PARÁMETROS DEL SCRIPT

Supón que ejecutas:

```
./script.sh hola 42 Ana
```

Código	Significado
\$0	Nombre del script
\$1..\$N	Parámetros posicionales
\$*	Todos los parámetros
\$#	Número de parámetros
shift	Desplaza los parámetros

ARRAYS

Declaración:

```
frutas=("manzana" "pera" "platano")
```

Acceso:

```
echo ${frutas[1]}
```

Total de elementos:

```
echo ${#frutas[@]}
```

Recorrer:

```
for f in "${frutas[@]}; do
echo $f;
done
```

Eliminar elemento:

```
unset frutas[1]
```

CONDICIONALES

```
if [ condición ]; then
    comandos
elif [ condición ]; then
    comandos
else
    comandos
fi
```

Comparaciones comunes

Números:

```
-eq, -ne, -lt, -le, -gt, -ge
```

Cadenas:

```
[ "$var" = "texto" ],  
-z (vacía),  
-n (no vacía)
```

Ficheros:

-e, -f, -d, -r, -w, -x, -s, -h

Negación:

```
if [ ! -d "$ruta" ]
```

AND / OR:

```
if [ $n -gt 0 -a $n -lt 10 ]
```

CASE (switch)

```
case $opcion in  
  a|A) comando1 ;;  
  b|B) comando2 ;;  
  *)    echo "Opción inválida" ;;  
esac
```

BUCLES

for

```
for i in `seq 1 5`; do  
  echo "Hola número $i"  
done
```

while

```
numero=0  
while [ $numero -le 10 ]; do  
  read -p "Introduce un número: "  
  numero  
done  
echo "Fin"
```

FUNCIONES

Definición y llamada:

```
saludar() {  
  echo "Hola!"  
}  
saludar
```

Con argumentos:

```
mostrar() {  
  echo "Hola $1, edad $2"  
}  
mostrar "Ana" 25
```

Devolver valor:

```
sumar() {  
  echo $(( $1 + $2 ))
```

```
}  
resultado=$(sumar 3 5)
```

Return con código:

```
verificar_par() {  
  (( $1 % 2 == 0 )) && return 0 ||  
  return 1  
}  
verificar_par 4  
[ $? -eq 0 ] && echo "Es par" ||  
echo "Es impar"
```

EJECUTAR UN SCRIPT

1. Dar permisos:

```
chmod +x script.sh
```

2. Ejecutar:

```
./script.sh  
# o sin permisos:  
bash script.sh
```